

Honest responses: status codes, errors & validation

Part A · Status codes

#	Do this	How
A1	Map every outcome	For POST/GET/PATCH/DELETE /orders, write the success status and the status for each real failure.
A2	Fix responses (if any)	Find every response that returns 200 for a failure and correct it — the status IS the result, never a 200 with { "ok": false }.
A3	Set Location on create	POST /orders returns 201 with Location: /orders/{id} pointing to the new order.
A4	Retry-safety	On 429/503 include Retry-After; state which endpoints are safe to retry (transient only; never a 4xx unchanged).

Part B · One error shape

#	Do this	How
B1	Standardise on Problem Details	Route every failure through your problem() helper so all errors share one shape: type / title / status / detail, with Content-Type: application/problem+json.
B2	Fix your type catalogue	Choose stable type strings and document them: empty-cart, item-unavailable, payment-declined, order-not-found, illegal-transition.
B3	Field-level validation errors	For validation failures return a 422 with an errors[] list naming each bad field and why.
B4	Don't leak internals	Wrap risky work in try/catch; log the detail server-side and return a mapped, clean error. Never send a stack trace or DB message to the client.

Part C · Request validation & formats

#	Do this	How
C1	Describe the body	Write a JSON Schema (or explicit checks) for the POST /orders body: items non-empty; each qty an integer ≥ 1 ; address required.
C2	Validate, then act	Validate before doing any work and collect all errors, not just the first. Invalid $\rightarrow$ one 422; valid $\rightarrow$ proceed.
C3	Content negotiation	Honour Accept: return application/json by default, application/xml if asked; 406 if you can't produce it.
C4	Label every body	Set Content-Type (+ charset=utf-8) on every response, errors included.

Part D · Prove it with curl

#	Do this	How
D1	Success transcript	curl -i the happy path; capture the 201, the Location header, and the JSON body.
D2	Failure transcript	Drive every failure — empty cart, qty ≤ 0 , unknown id, illegal transition, rate limit — and capture each status + Problem Details body.
D3	Negotiation check	Request an order with Accept: application/xml and capture the response (XML, or 406).

#	Do this	How
D4	No regressions	Re-run your A5 checks (idempotency, timeouts) and confirm they still hold.

WHAT TO SUBMIT

- ▶ Your updated service source — built on A5, the **same** Orders service (no new service).
- ▶ `openapi.yaml` updated: the status codes and the Problem Details error schema for each endpoint.
- ▶ `transcript.txt` — a curl transcript covering the success path and every failure path.
- ▶ `NOTES.md` with the answers below.
- ▶ One submission per Team via Google Classroom, as a single zip. Roll numbers and names of all members on the first page.

ANSWER THESE IN NOTES.MD

1. Your status map: every endpoint, its success status, and the status for each failure — one line on why each.
2. Your type catalogue: the stable string for each failure and what triggers it.
3. Paste one Problem Details body from your service and label its fields.
4. Show a field-level 422 that reports two bad fields at once.
5. Which endpoints are safe to retry, and which status codes tell a client so?
6. Where did you map an internal error to a clean client error, and what did you hide from the caller?
7. (If done) Show the same order as JSON and as XML, and the Accept header that chose each.